© 2026 ANSYS, Inc. or affiliated companies
Unauthorized use, distribution, or duplication prohibited.

ansys-pyetk

ANSYS, Inc.
Southpointe
2600 Ansys Drive
Canonsburg, PA 15317
ansysinfo@ansys.com
http://www.ansys.com
(T) 724-746-3304
(F) 724-514-9494

Mar 03, 2026

ANSYS, Inc. and ANSYS Europe, Ltd. are UL registered ISO 9001:2015 companies.

CONTENTS

Useful links: *Installation* | Source repository | Issues

The Electronic Transformer toolKit (PyETK) is a custom solution for electronic transformer design.

Getting started Learn how to install the Electronic Transformer toolKit (PyETK), understand its architecture and use this toolkit's common methods to develop electronic transformer models.

Getting started User interface How to use the user interface.

User Interface API reference Understand the APIs available for the PyETK.

API reference

CHAPTER

ONE

GETTING STARTED

This section explains how to install the Electronic Transformer toolKit (PyETK). It then explains the general terminology used and provides an example of how to use its common methods.

Installation Learn how to install the PyAEDT Common Toolkit.

Installation Dimensioning PyETK's Transformer Build Reference.

Dimensioning Examples Basic Examples.

Examples

1.1 Installation

1.1.1 Download links

The following installers are available for different operating systems:

Table 1: Available Installers

Installer Link	Operating System
Windows Installer	Windows
Ubuntu 22.04 Installer	Ubuntu 22.04
Ubuntu 24.04 Installer	Ubuntu 24.04

1.2 Installing the `Electronic Transformer toolKit (PyETK)`

Windows

First step is installing the `PyETK`. In order to do so, follow the next steps.

1. Download the necessary installer from the latest available release. The file should be named `PyETK-Installer-Windows.exe`.
2. Execute the installer.
3. Search for the `PyETK` and run it.

The `PyETK` window should appear at this stage.

Linux

Ubuntu

Prerequisites:

1. **OS** supported for **Ubuntu(24.04 and 22.04)**.
2. Update `apt-get` repository and install the following packages with **sudo** privileges: **wget, gnome, libffi-dev, libssl-dev, libsqlite3-dev, libxcb-xinerama0 and build-essential** packages with **sudo** privileges

```
sudo apt-get update -y
sudo apt-get install wget gnome libffi-dev libssl-dev libsqlite3-dev libxcb-
↪xinerama0 build-essential -y
```

3. Install **zlib** package

```
wget https://zlib.net/current/zlib.tar.gz
tar xvzf zlib.tar.gz
cd zlib-*
make clean
./configure
make
sudo make install
```

To install the `Template Toolkit`, follow below steps.

1. Download the necessary installer from the latest available release. The file should be named `Toolkit-Template_*.zip`.
2. Execute the below command on the terminal

```
unzip Toolkit-Template_*.zip
./installer.sh
```

3. Search for the `PyETK` and run it.

The `PyETK` window should appear at this stage.

To uninstall the `PyETK`, follow below steps.

1. Go to File menu. Click Uninstall option.
2. Click `Uninstall` button.

1.3 Python installation

The Toolkit Template can be installed like any other open source package.

From PyPI, you can either install both the backend and user interface (UI) methods or install only the backend methods.

To install both the backend and UI methods, run this command:

```
pip install ansys-pyetk[all]
```

If you only need the common API, install only the backend methods with this command:

```
pip install ansys-pyetk
```

1.4 For developers

You can be up and running with four lines of code:

```
git clone https://github.com/ansys/ansys-pyetk
cd ansys-pyetk
python -m pip install -U pip uv
uv venv
uv pip install -e .
```

Now you can run it with:

```
uv run run_toolkit
```

Details

Installing Pytools installer in developer mode allows you to modify the source and enhance it.

Before contributing to the project, please refer to the PyAnsys Developer's guide. You need to follow these steps:

1. Start by cloning this repository:

   ```
   git clone https://github.com/ansys/ansys-pyetk
   ```

2. Create a fresh-clean Python environment and activate it. Refer to the official venv documentation if you require further information:

   ```
   # Create a virtual environment
   python -m uv venv .venv

   # Activate it in a POSIX system
   source .venv/bin/activate

   # Activate it in Windows CMD environment
   .venv\Scripts\activate.bat

   # Activate it in Windows Powershell
   .venv\Scripts\Activate.ps1
   ```

3. Make sure you have the latest version of pip:

   ```
   python -m pip install -U pip uv
   ```

4. Install the project in editable mode:

   ```
   python -m uv pip install -e .[tests,doc]
   ```

5. Finally, verify your development installation by running:

   ```
   uv run pytest tests -v
   ```

1.4.1 Style and testing

This project uses pre-commit. Install with:

```
uv pip install pre-commit
uv run pre-commit install
```

This now runs `pre-commit` for each commit to ensure you follow project style guidelines. For example:

```
git commit -am 'fix style'
isort....................................................................Passed
black....................................................................Passed
blacken-docs.............................................................Passed
flake8...................................................................Passed
codespell................................................................Passed
pydocstyle...............................................................Passed
check for merge conflicts................................................Passed
debug statements (python)................................................Passed
check yaml...............................................................Passed
trim trailing whitespace.................................................Passed
Validate GitHub Workflows................................................Passed
```

If you need to run it again on all files and not just staged files, run:

```
uv run pre-commit run --all-files
```

1.4.2 Local build

This application can be deployed as a 'frozen' application using pyinstaller with:

```
uv pip install -e .[freeze]
uv run pyinstaller frozen.spec
```

This generates application files at `dist/ansys_python_manager` and you can run it locally by executing `Ansys Python Manager.exe`.

1.4.3 Documentation

For building documentation, you can either run the usual rules provided in the Sphinx Makefile:

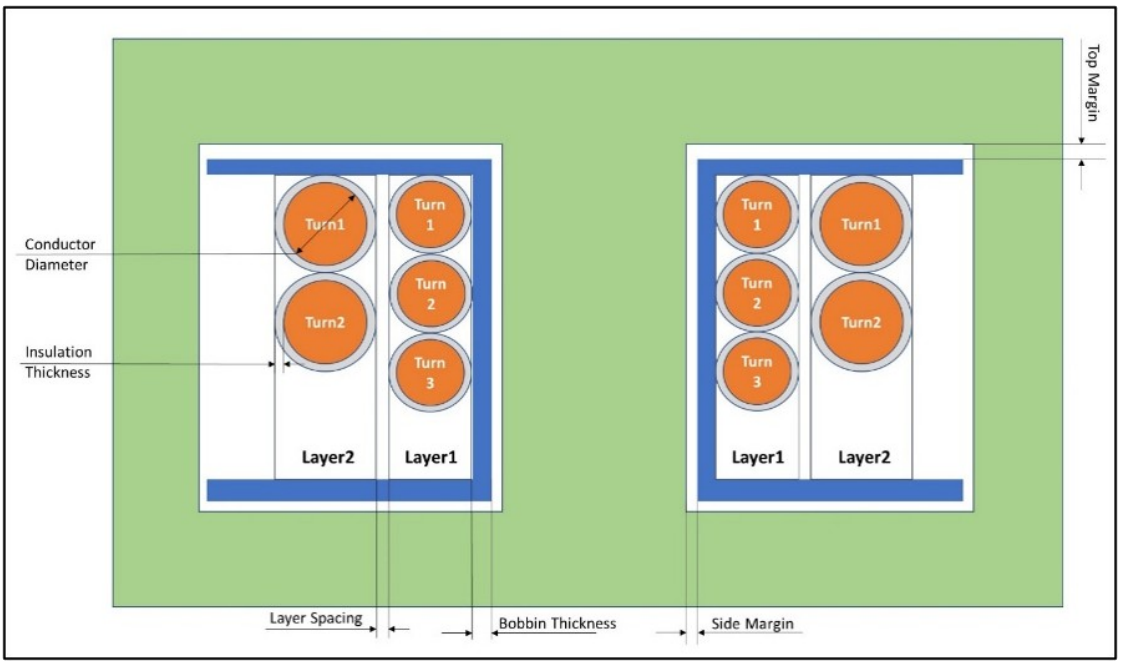
```
uv pip install -e .[doc]
uv run make -C doc/ html

# subsequently open the documentation with (under Linux):
<your_browser_name> doc/html/index.html
```

1.5 Dimensioning

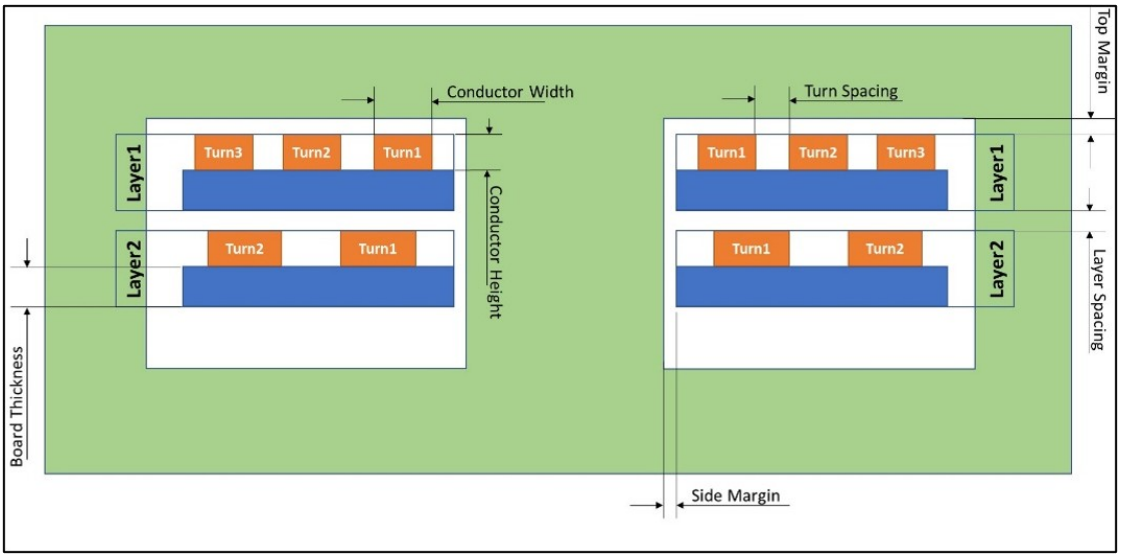
To get the most out of PyETK, the key dimensions used throughout the platform are displayed in this document. Understanding these streamlines the workflow and ensure the use of every feature to its full potential.

The transformer dimensions - from core to turn distances can be observed in two main build topologies - the planar and wound transformer. The main difference between the two is that the planar transformer's winding is embedded in the PCB as traces, while the wound winding revolves around a leg of the magnetic core.

1.5.1 Wound build type



1.5.2 Planar build type



1.6 Examples

Examples are available to demonstrate how to use pyETK in different ways.

Using the Backend API. Example of how to use the pyETK API to create an electronic transformer.

Use the ToolkitBackend API to create a transformer model in AEDT

Using the User Interface. Example of how to use the pyETK's UI to create an electronic transformer.

Create transformer using the user interface

Migrate from ETK to PyETK. Using the User Interface to migrate ACT's json files to PyETK.

From ACT to PyETK example

Copyright (C) 2023 - 2026 ANSYS, Inc. and/or its affiliates. SPDX-License-Identifier: Apache-2.0

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

```
http://www.apache.org/licenses/LICENSE-2.0
```

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

1.6.1 Use the ToolkitBackend API to create a transformer model in AEDT

This example demonstrates how to use the `ToolkitBackend` class. More specifically, it does the following:

- Initializes the `ToolkitBackend` class.
- Sets common properties like `non_graphical` and `aedt_version`.
- Defines the path to the JSON file with the transformer properties.
- Reads the properties from the JSON file.
- Launches AEDT.
- Creates the transformer model.

[1]:
```
# ## Perform required imports
from pathlib import Path
import sys
import tempfile
```

[2]:
```
from ansys.aedt.core import generate_unique_project_name
from ansys.aedt.toolkits.electronic_transformer.backend.api import ToolkitBackend
```

```
/home/runner/work/ansys-pyetk/ansys-pyetk/.venv/lib/python3.10/site-packages/ansys/aedt/
↪core/internal/aedt_versions.py:67: UserWarning: No installed versions of AEDT are␣
↪found in the system environment variables ``ANSYSEM_ROOTxxx`` or``AWP_ROOTxxx``.
  warnings.warn(
```

[3]:
```
# ## Create backend instance
toolkit_api = ToolkitBackend()
```

[4]:
```
# ## Set common properties
toolkit_api.properties.non_graphical = False
toolkit_api.properties.aedt_version = "2025.2"
```

[5]:
```
# ## Define the JSON path
json_file_name = "RM_wound_circular.json"
temp_dir = tempfile.TemporaryDirectory(suffix="_ansys")
project_name = generate_unique_project_name(root_name=temp_dir.name, project_name="ETK_
↪example")
try:
    # Works in standard .py scripts
    json_path = Path(__file__).resolve().parents[3]
```

(continues on next page)

(continued from previous page)

```
except NameError:
    # Fallback for Jupyter / Interactive sessions
    json_path = Path.cwd().parents[2]
json_path = Path(json_path) / "tests" / "backend" / "json_files" / json_file_name
```

[6]:
```
# ## Set JSON reading properties
toolkit_api.load_properties_from_json(json_path)
```

[7]:
```
# ## Launch AEDT
thread_msg = toolkit_api.launch_thread(toolkit_api.launch_aedt)
idle = toolkit_api.wait_to_be_idle()
if not idle:
    print("AEDT not initialized.")
    sys.exit()
```

```
PyAEDT INFO: Python version 3.10.19 (main, Oct 10 2025, 01:01:37) [GCC 13.3.0].
PyAEDT INFO: PyAEDT version 0.25.1.
PyAEDT INFO: Initializing new Desktop session.
```

```
Exception in thread Toolkit_Thread:
Traceback (most recent call last):
  File "/opt/hostedtoolcache/Python/3.10.19/x64/lib/python3.10/threading.py", line 1016,␣
↪in _bootstrap_inner
    self.run()
  File "/opt/hostedtoolcache/Python/3.10.19/x64/lib/python3.10/threading.py", line 953,␣
↪in run
    self._target(*self._args, **self._kwargs)
  File "/home/runner/work/ansys-pyetk/ansys-pyetk/.venv/lib/python3.10/site-packages/
↪ansys/aedt/toolkits/common/backend/thread_manager.py", line 55, in process_exe
    process(*args)
  File "/home/runner/work/ansys-pyetk/ansys-pyetk/.venv/lib/python3.10/site-packages/
↪ansys/aedt/toolkits/common/backend/api.py", line 429, in launch_aedt
    self.desktop = ansys.aedt.core.Desktop(**desktop_args)
  File "/home/runner/work/ansys-pyetk/ansys-pyetk/.venv/lib/python3.10/site-packages/
↪ansys/aedt/core/desktop.py", line 666, in __init__
    self.__check_version(version, student_version)
  File "/home/runner/work/ansys-pyetk/ansys-pyetk/.venv/lib/python3.10/site-packages/
↪ansys/aedt/core/desktop.py", line 2421, in __check_version
    raise AEDTRuntimeError("AEDT is not installed on your system. Install AEDT version␣
↪2022 R2 or higher.")
ansys.aedt.core.internal.errors.AEDTRuntimeError: AEDT is not installed on your system.␣
↪Install AEDT version 2022 R2 or higher.
ERROR - Toolkit has crashed and is not functional.
```

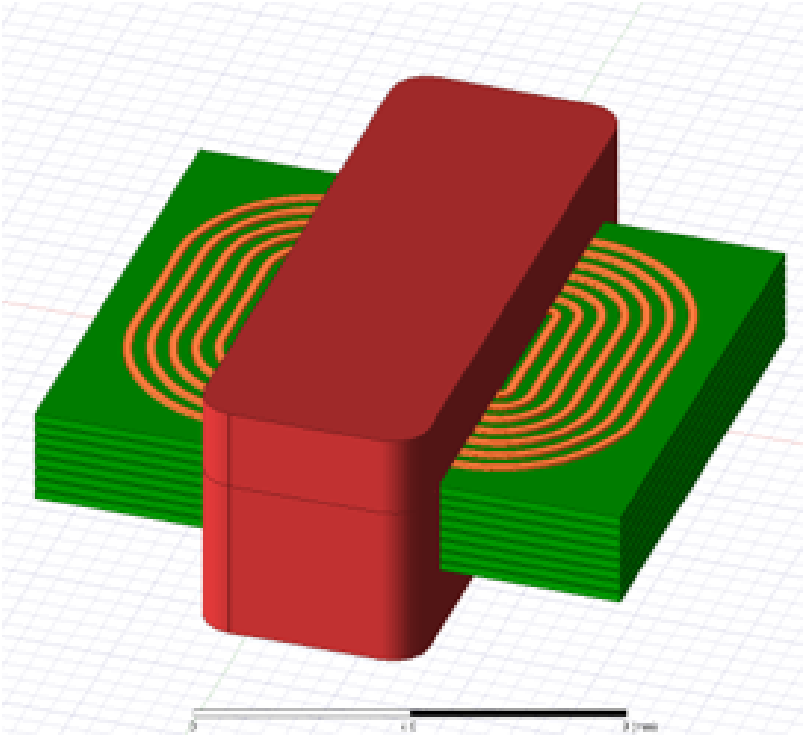
[8]:
```
# ## Create the models
setup = toolkit_api.create_model()
```

```
ERROR - Process ID is not defined.
ERROR - Not connected
```

1.6.2 Create transformer using the user interface

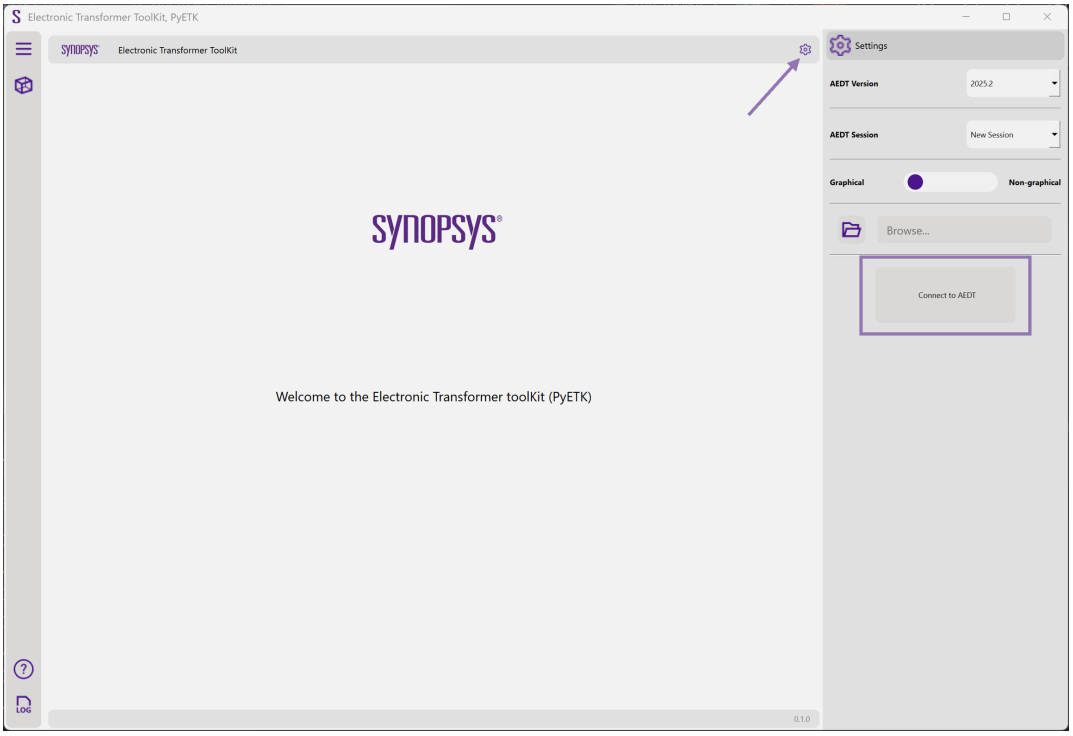
This example demonstrates how to use the PyETK's User Interface (UI) to create an electronic transformer. It provides a step-by-step guide on how to navigate through the UI, specify settings, and design a transformer using the available fields and tools.

The design is that of a planar transformer, which is commonly used in power electronics applications.

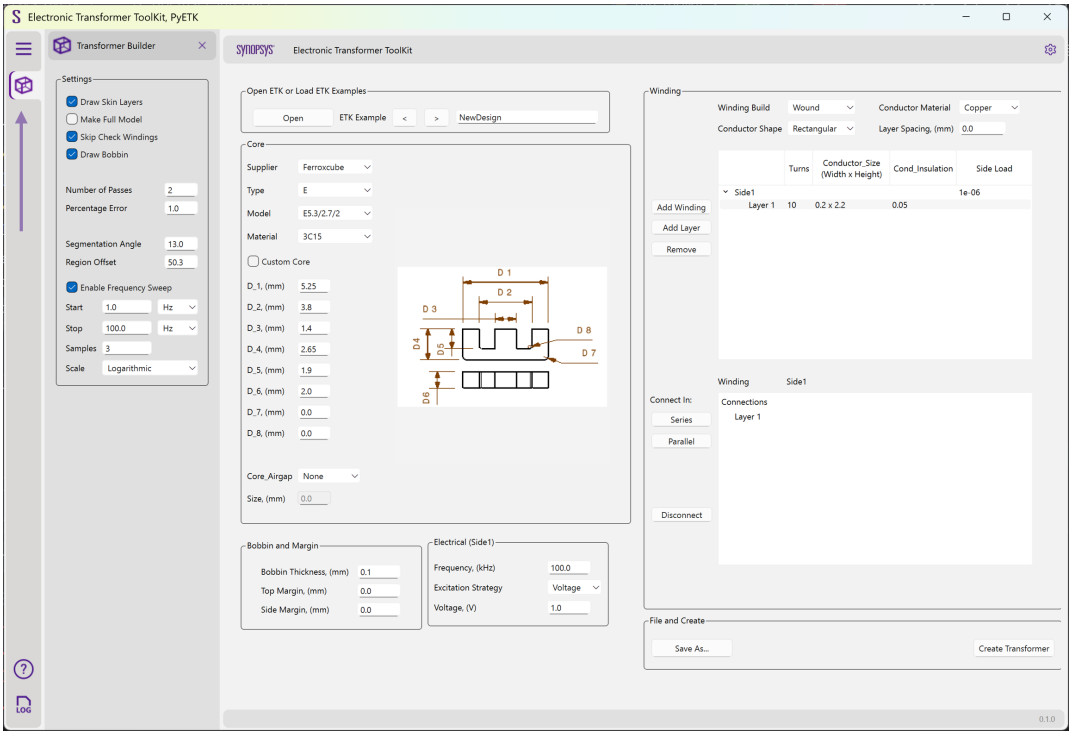


For simplicity, the example focuses on one sub-menu at a time, starting with the core model, dimensions, and materials. Then, moving to the bobbin, winding, and excitation information, and finally, the Maxwell settings.

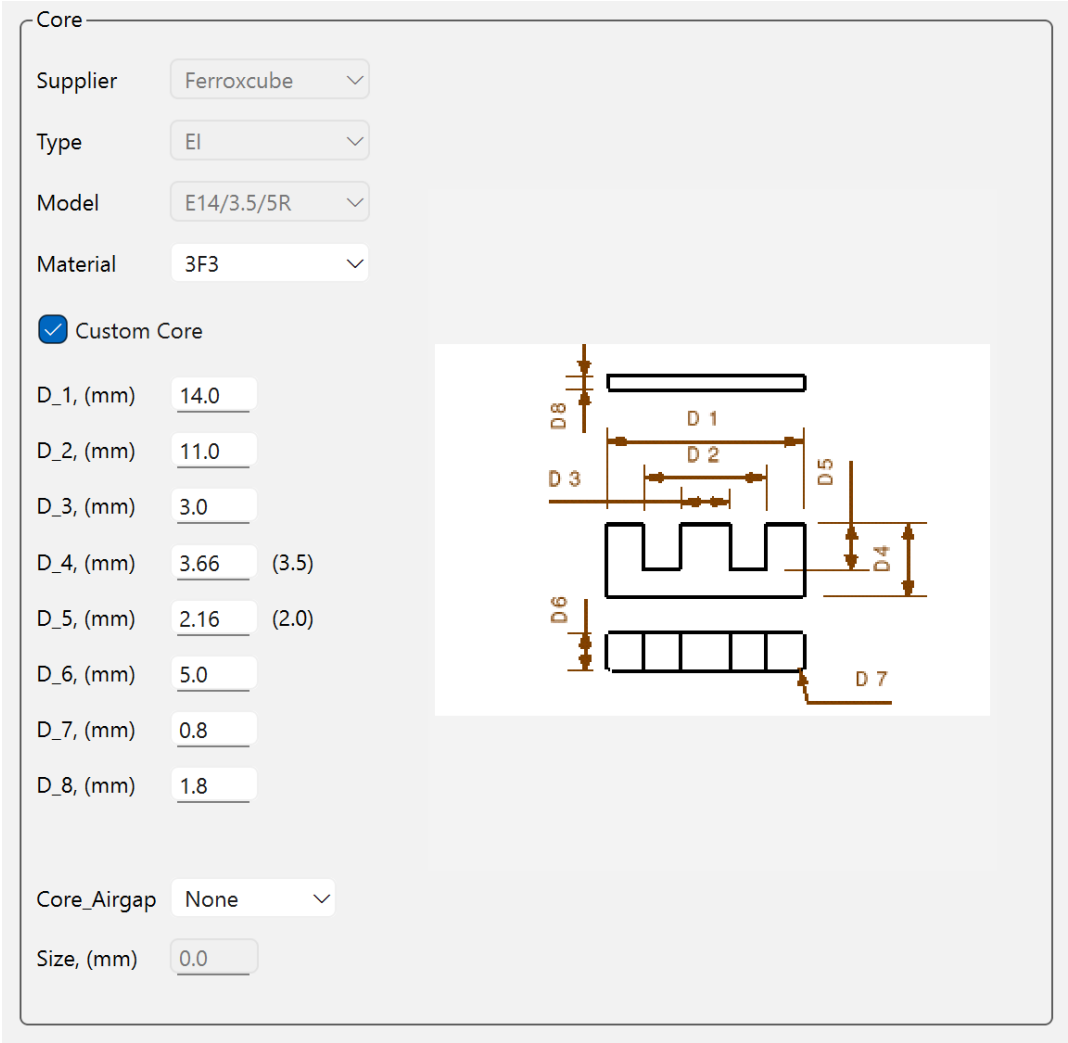
1. First, launch the PyETK, and connect to AEDT by clicking the **Connect to AEDT** button under the settings tab. For installation instructions, see *Installation*.



2. Next, navigate to the **Transformer Builder** tab. This tab contains all the fields and tools to design and model electronic transformers.



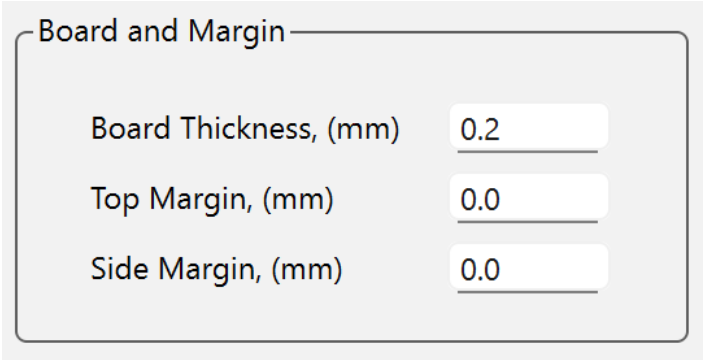
3. Modify the core information in the **Core** section by selecting the core type, core dimensions, and core material.



Note

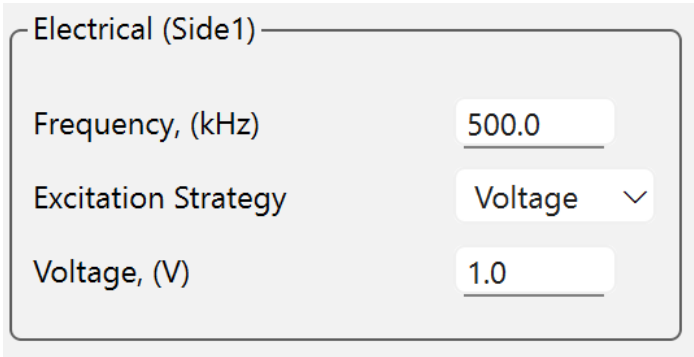
The **Custom Core** checkbox enables the user to use a standard core shape with custom dimensions.

4. Then, modify the board information in the **Bobbin and Margin** section by selecting the bobbin type and bobbin dimensions.



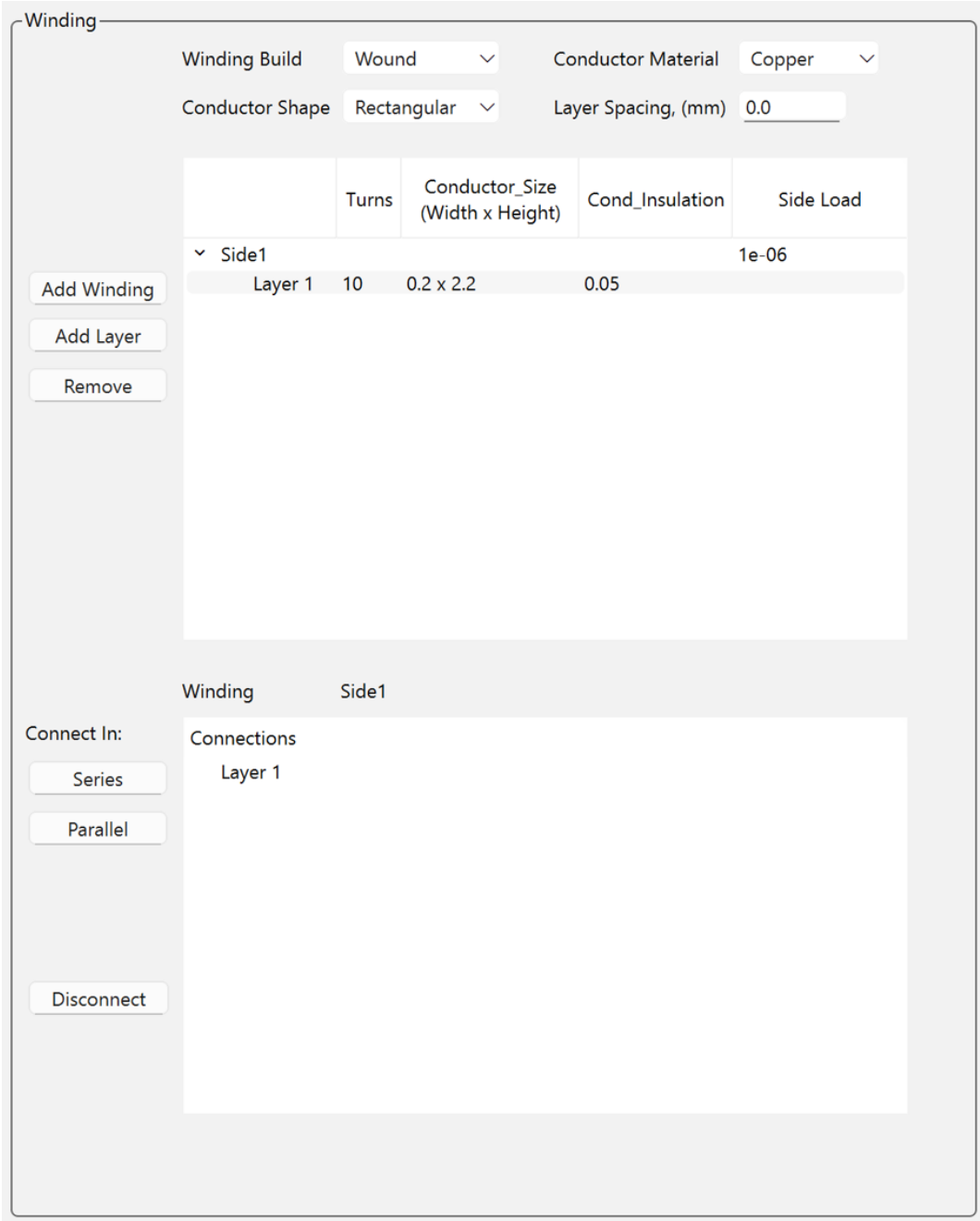
> **Note**
>
> In PyETK the names bobbin and board are interchangeable. **Bobbin** is used to refer to *Wound* build types, while **Board** is used to refer to *Planar* build types. The fields in the UI are the same for both build types.

5. Modify the electrical source information such as the excitation type, excitation value, and frequency.



6. Create the transformer windings layer by layer by specifying the build type, material, number of turns, and turn spacing for each layer in the **Winding** section.

    - The **Winding** section is initialized as follows:

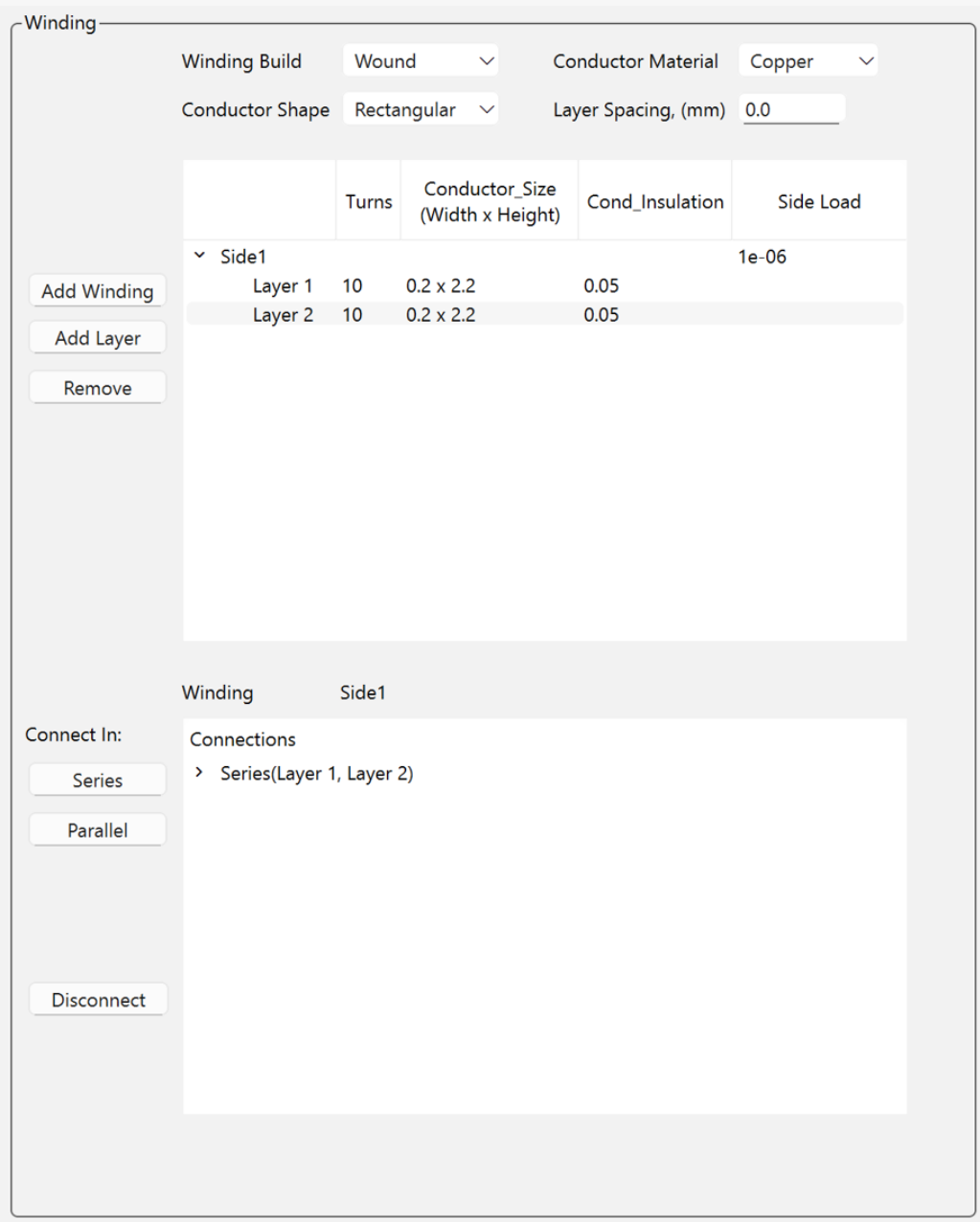


Warning

The Transformer cannot be created without at least a single winding layer. Hence, the initialization of the winding section contains one layer with default values. The user can modify the default values or add more layers as needed. In addition, layer connection must be specified. By default, every layer in a winding side is connected in **Series**

To modify the circuit connections between layers, the user must select the layers of interest, click the **Disconnect** button and then specify either **Series** or **Parallel** connection. The user can also specify the connection between the primary and secondary sides by selecting the **Primary-Secondary Connection** button and selecting either **Series** or **Parallel** connection between them.

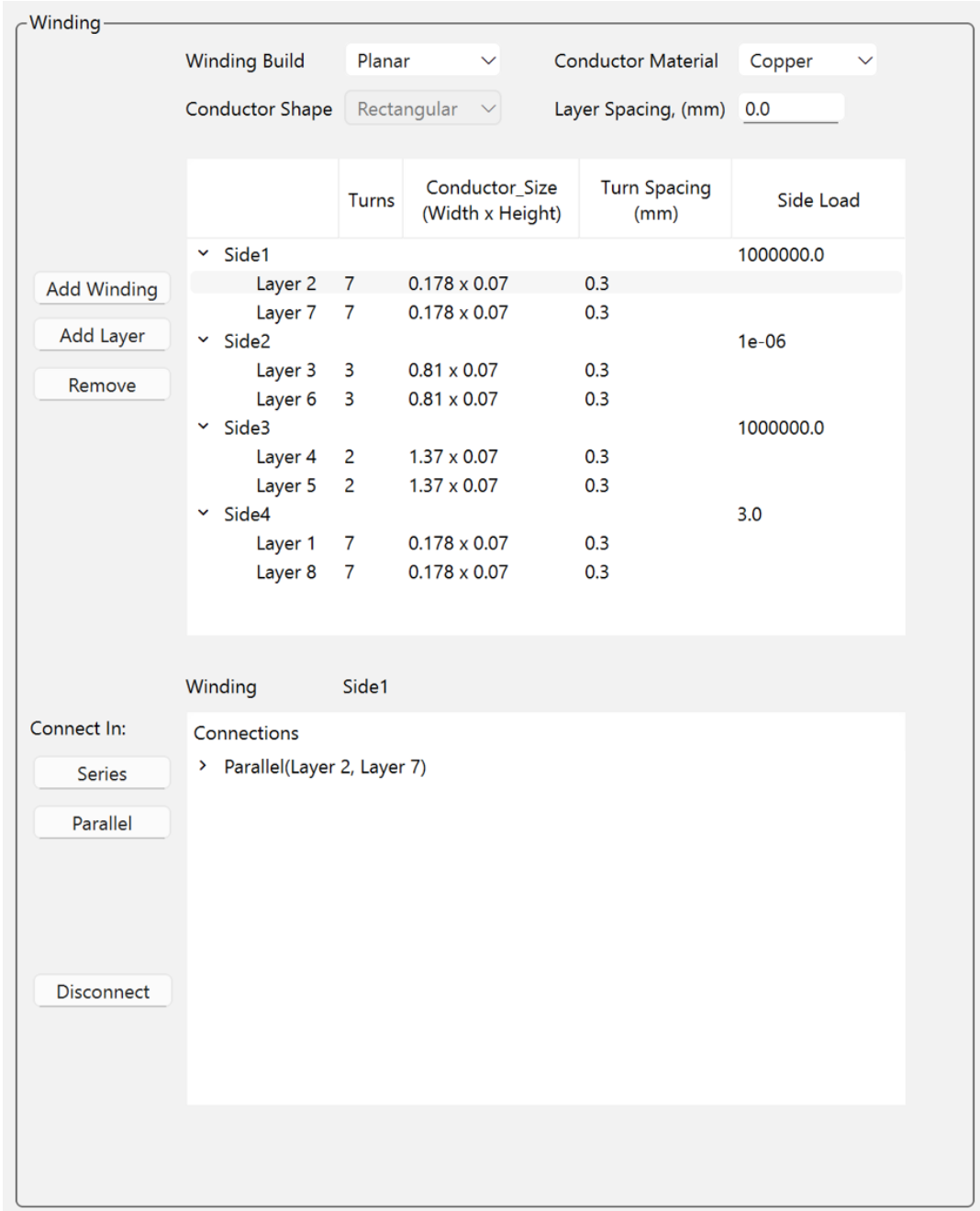
See image below for more details.



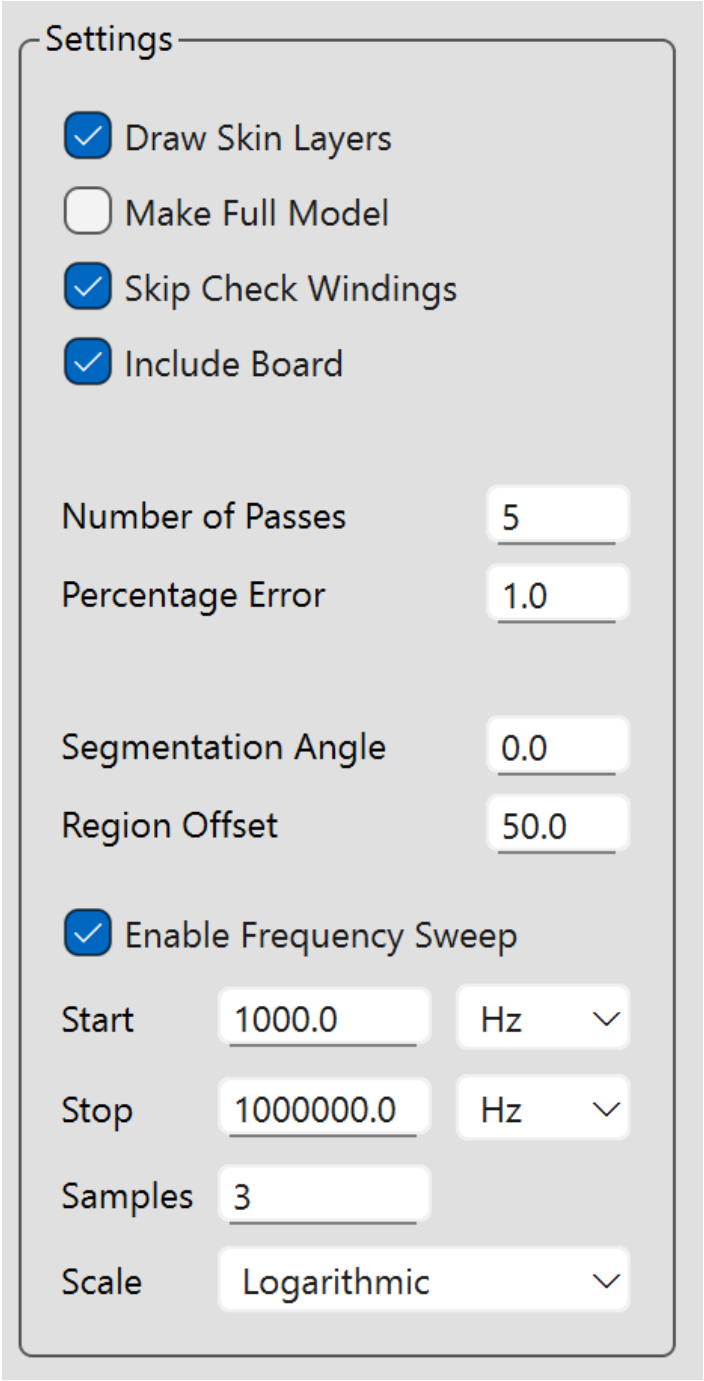
After disconnecting the default connection

Connecting the layers in parallel

- Create the winding layers and connections for the planar transformer as follows:



7. Specify the Maxwell settings such as the number of passes, percent error, maximum number of passes, and frequency sweeps.



Note

The Maxwell settings are used to specify the settings for the Maxwell analysis that is performed after creating the transformer geometry in AEDT. These settings are used to control the accuracy and convergence of the Maxwell analysis.

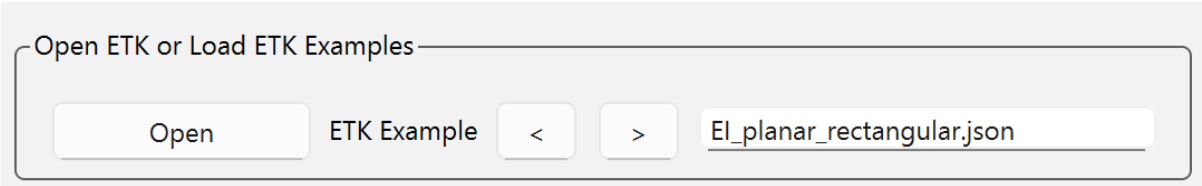
8. Finally, click **Create Transformer** to create the transformer geometry in AEDT. The log window provides information about the creation process, including any errors or warnings that may occur during the process.

Pre-packaged examples - Loading a transformer model from a .json file

To enable third party integration, PyETK consumes a transformer definition defined in a versioned .json file. This .json contains all the information including sources, dimensions, materials, and other advanced settings. The .json configuration file can be used with PyETK's API, bypassing the need for the UI.

The .json file can be created by saving the current transformer model in the UI using the **Save As** button, or it can be created manually by following the structure of the .json file.

To browse through the pre-packaged examples, you have two options either click the **Open** button, or navigate using the forward and backward buttons as shown in the image below.



Note

If you were an ACT ETK user, you can load your ACT .json file in the PyETK UI by clicking on the **Open** button and selecting your ACT .json file. The PyETK automatically parses the ACT .json file and populate the fields in the UI with the corresponding values from the ACT .json file. After loading the ACT .json file, you can review the populated fields in the UI to ensure that all the information has been correctly transferred. If you're planning on reusing the configuration file, click *Save As*. This way the ACT .json file is saved in the latest working .json format. See act_to_pyetk_example for more details on migrating from ACT to PyETK.

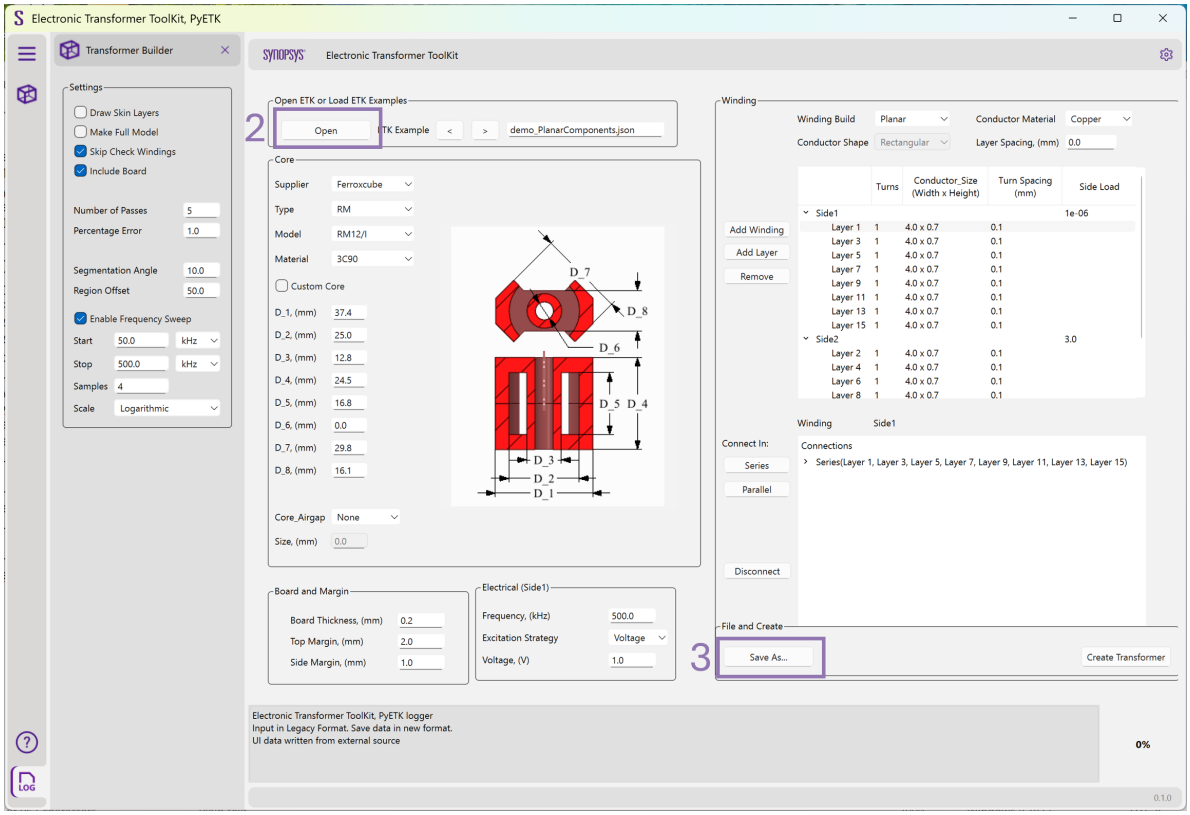
1.6.3 From ACT to PyETK example

This example demonstrates how to migrate from the ACT (Ansys Customization Toolkit) to the PyETK. It provides a step-by-step guide on how to transition your existing ACT .json files to PyETK.

1. First, ensure you have a working ACT .json file. This file contains the configuration and settings for your transformer. The ACT's ETK .json file has no version and its structure contains three main sections: `core_dimensions`, `winding_definitions` and `setup_definitions`. It looks similar to the following:

```
{
  "core_dimensions": {
    "segmentation_angle": 0,
    "supplier": "Ferroxcube",
    "core_type": "EI",
    "core_model": "E14/3.5/5R",
    "dimensions": {
    "D_1": "14.0",
    "D_2": "11.0",
    "D_3": "3.0",
    "D_4": "3.66",
    "D_5": "2.16",
    "D_6": "5.0",
    "D_7": "0.8",
    "D_8": "1.8"
    },
    "airgap": {
      "define_airgap": false
    }
  },
  "winding_definition": {
    "material_name": "Copper",
```

2. Next, open the PyETK User Interface (UI) and navigate to the **Transformer Builder** tab. Click *Open* and select your ACT .json file to load it into the PyETK. The PyETK automatically parses the ACT .json file and populate the fields in the UI with the corresponding values from the ACT .json file.
3. After loading the ACT .json file, you can review the populated fields in the UI to ensure that all the information has been correctly transferred. If you're planning on reusing the configuration file, click *Save As*. This way the ACT .json file is saved in the latest working .json format.



Note

The Log window provides information about the loading process, including any errors or warnings that may occur during the migration.

4. PyETK's versioned .json file looks similar to the original ACT .json configuration file, however the differences enable better organization that allows for better readability and easier access to the different sections of the transformer configuration. The PyETK .json file contains a version number, which helps to track changes and updates to the file format over time. The PyETK .json file also has a more structured format, with clear sections for core dimensions, winding definitions, and setup definitions. This structure allows for easier navigation and understanding of the different components of the transformer configuration. Furthermore, it's versioned which implies that it can be updated and improved over time – including further feature development – without breaking compatibility with existing files. The versioned .json file looks like the following:

_static/pyetk-toolkit-json.png

CHAPTER

TWO

USER INTERFACE

This section describes how to use the PyETK. It assumes that you have already launched the toolkit. For toolkit installation, see *Installation*

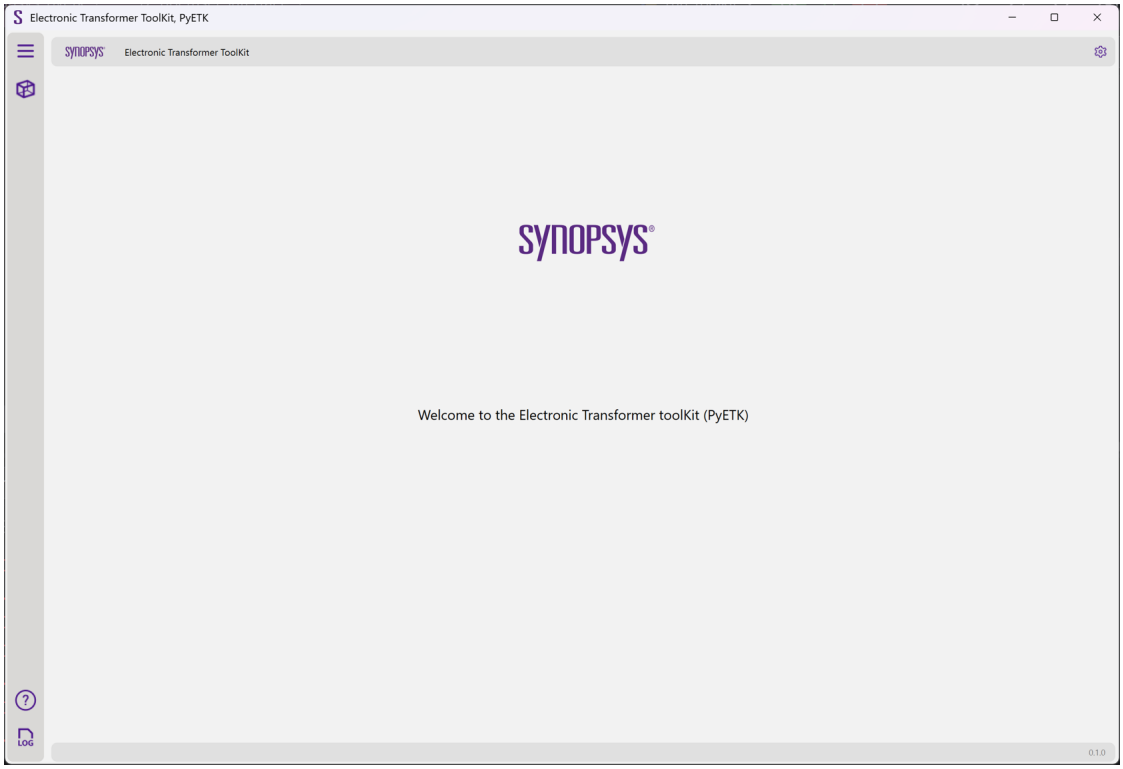
The first section focuses on the general tools available in the UI, coming from the common toolkit framework. The second section focuses on the application specific fields and tools to design and model electronic transformers.

CHAPTER

THREE

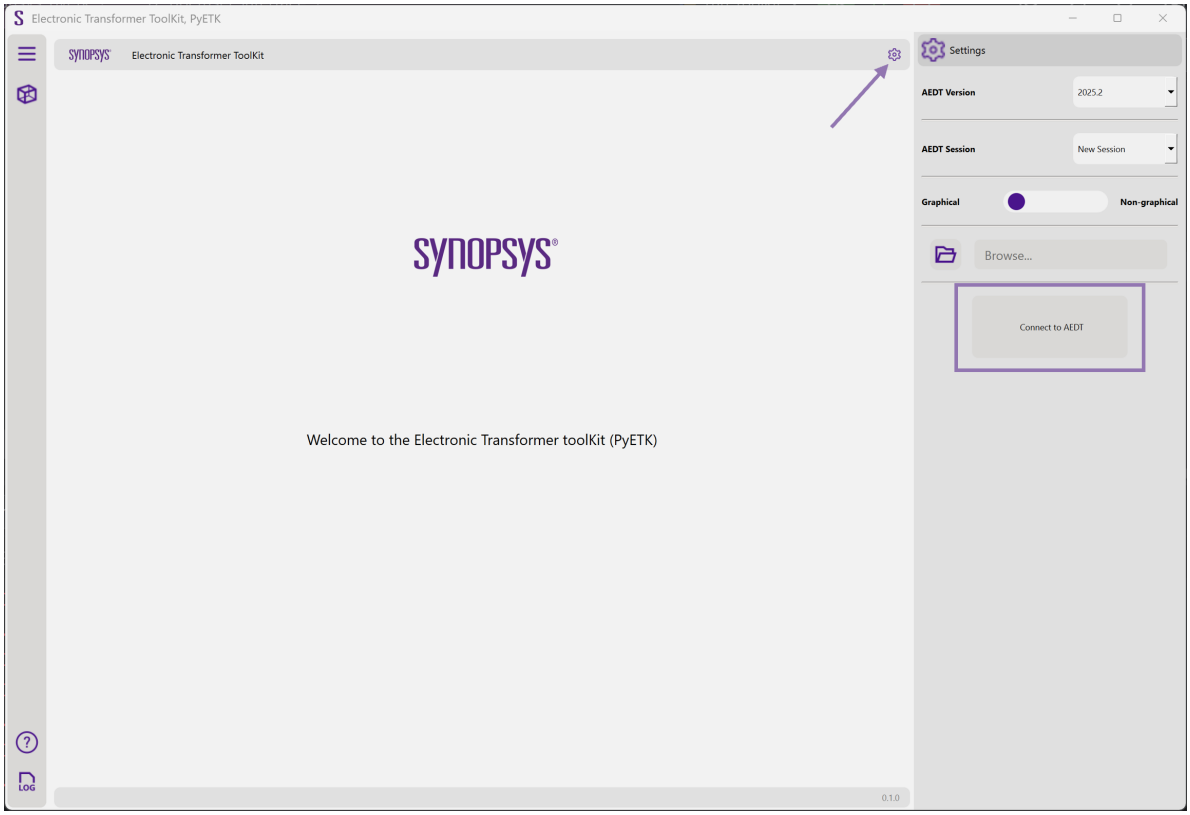
TOOLKIT COMMON FRAMEWORK

This section describes the use of the common framework of the PyETK, which is shared across all toolkits. Basic navigation through the UI starting from the starting page to the settings and log tabs is described below.

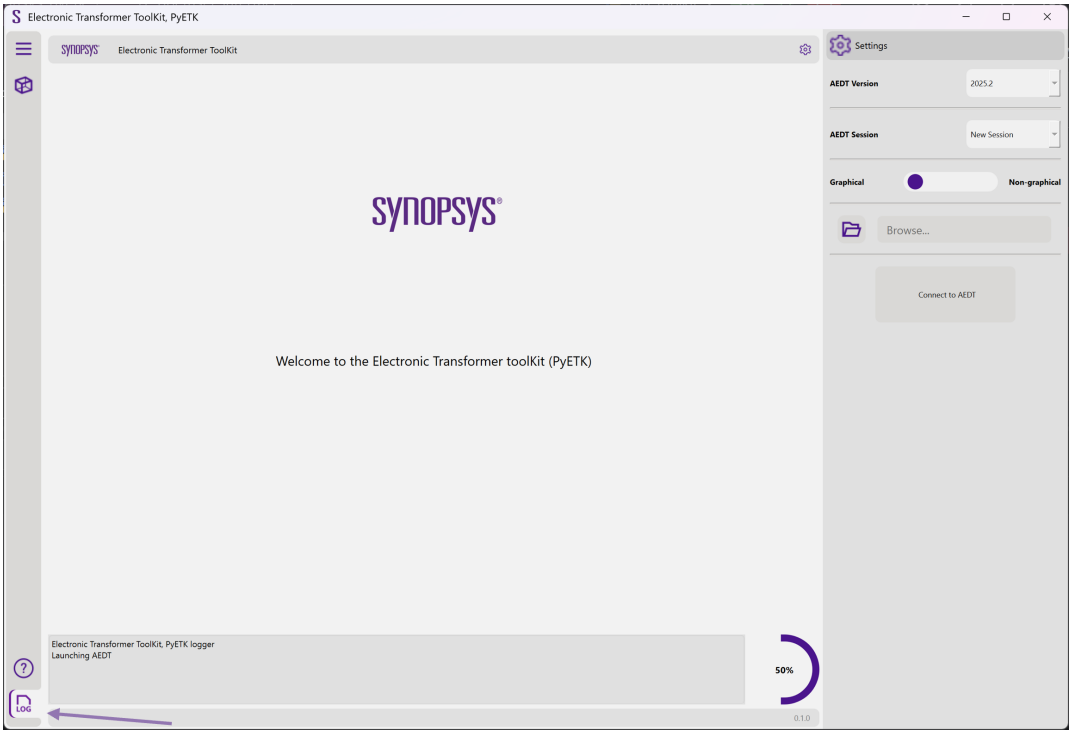


1. On the **Settings** tab, specify settings for either creating an AEDT session or connecting to an existing AEDT session.

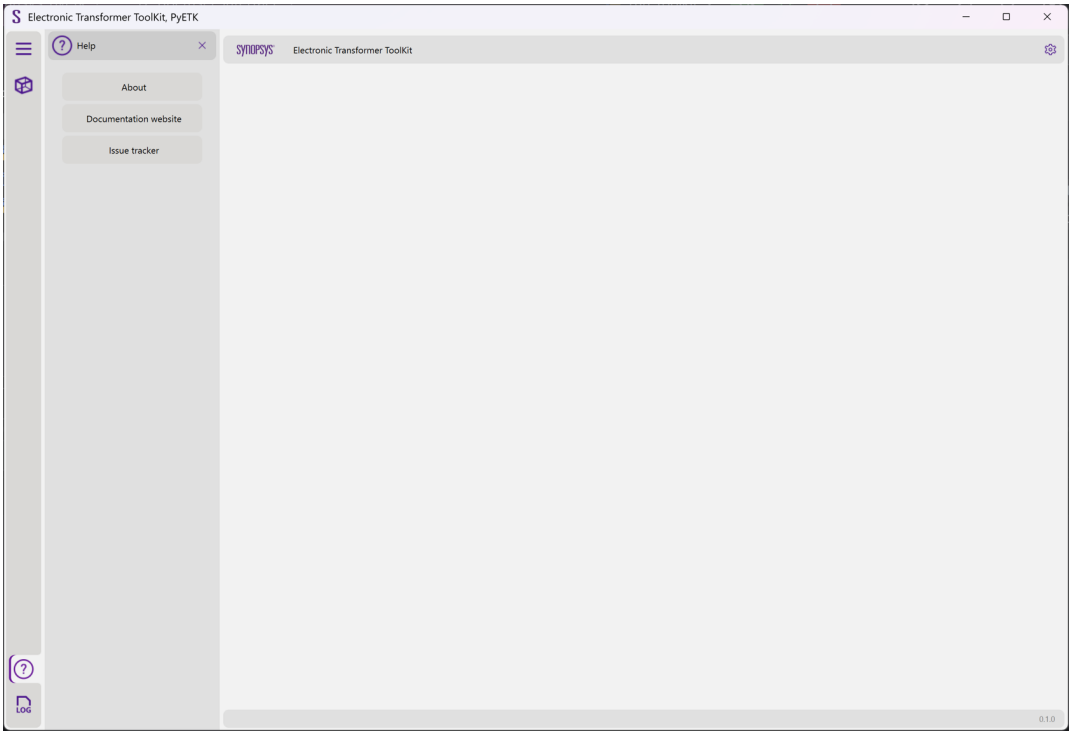
> **Note**
>
> If the PyETK Wizard is launched from AEDT, the **Settings** tab does not appear because the toolkit is directly connected to the specific AEDT session.



2. A general log window is available by clicking on the **Log** tab on the left side. This log file provides information about the toolkit's operations, such as the status of the connection to AEDT, the creation of geometries, and any errors that may occur during the process.



3. The **Help** tab provides access to the PyETK documentation, and Github's tracked issues.

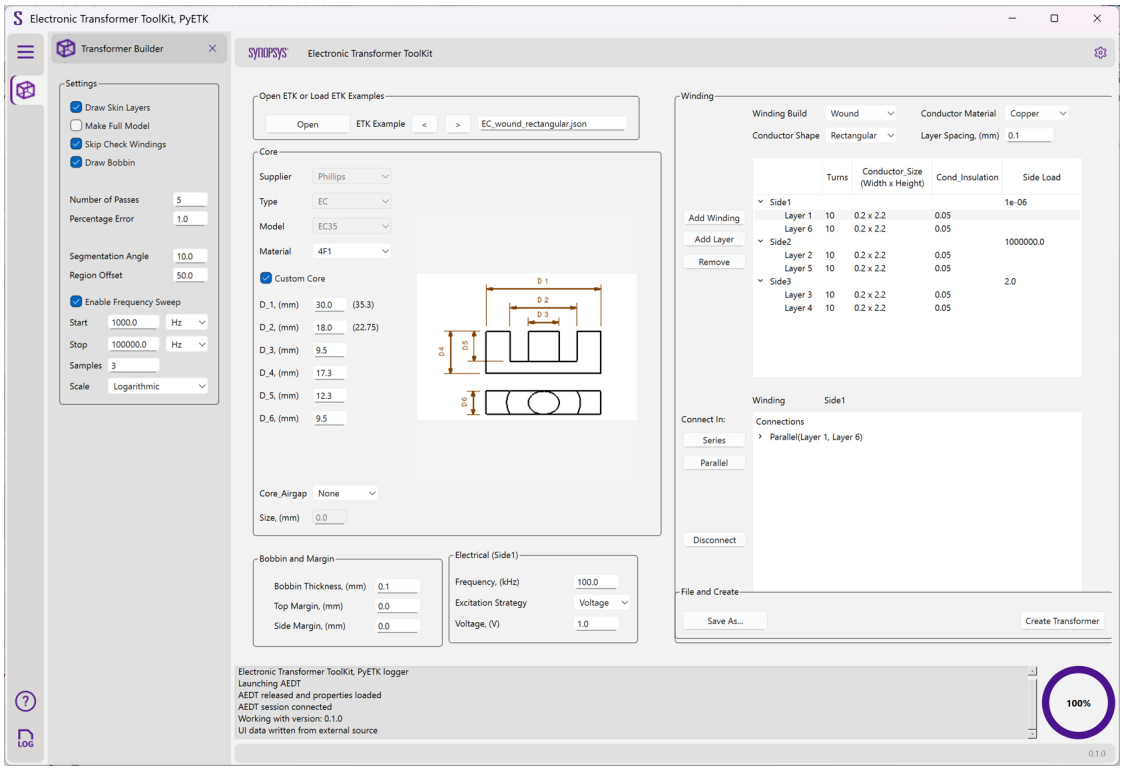

Electronic Transformer ToolKit, PyETK
Help
About
Documentation website
Issue tracker
SYNOPSYS
Electronic Transformer ToolKit
LOG
0.1.0

CHAPTER

FOUR

TRANSFORMER BUILDER

The **Transformer Builder** expands the fields to design and model electronic transformers. It provides fields to specify the core type, core dimensions, bobbin type, bobbin dimensions, winding information, and excitation information. In addition there is a settings section to specify Maxwell analysis settings such as number of passes, percent error, maximum number of passes and frequency sweeps.



CHAPTER

FIVE

API REFERENCE

The PyETK API contains the `ToolkitBackend` class, which provides methods for creating the whole model or some of the geometries, such as core, winding, bobbin, to help the user to visualize those parts only to create core types, bobbins, excitations, circuits, and ultimately put it all together as an electronic transformer model. You use the PyETK API at the toolkit level.

The common methods for creating an AEDT session or connecting to an existing AEDT session are provided by the Common toolkit library.

You can use the Toolkit API as shown in this example: *ETK API example*

class `ansys.aedt.toolkits.electronic_transformer.backend.api.`**`ToolkitBackend`**

Bases: `AEDTCommon`

Control the toolkit workflow.

This class provides methods to connect to a selected design and create the model or the parts of the transformer.

Examples

```
>>> from ansys.aedt.toolkits.electronic_transformer.backend.api import␣
↪ToolkitBackend
>>> toolkit_api = ToolkitBackend()
>>> toolkit_api.launch_aedt()
>>> toolkit_api.wait_to_be_idle()
>>> toolkit_api.create_core_geometry()
```

create_bobbin_geometry(*frontend_properties=None*)

Create the bobbin geometry.

Parameters

frontend_properties
[`dict`, `optional`] Frontend properties. The default is `None`.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.bobbin.Bobbin
Bobbin object.

create_core_geometry(*frontend_properties=None*)

Create the core geometry.

Parameters

frontend_properties
[`dict`, `optional`] Frontend properties. The default is `None`.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.core.Core
Core object.

create_model(*frontend_properties=None*)

Create the complete model.

Parameters

frontend_properties
[dict, optional] Frontend properties. The default is None.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.etk.ETK
ETK object.

create_winding_geometry(*frontend_properties=None*)

Create the winding geometry.

Parameters

frontend_properties
[dict, optional] Frontend properties. The default is None.

Returns

ansys.aedt.toolkits.electronic_transformer.backend.workflows.winding.Winding
Winding object.

load_properties_from_json(*file_path*)

Load properties from a JSON file.

Parameters

file_path
[str] Path to the JSON file.

validate_json(*data*)

Validate input data from a JSON file.

Parameters

data
[pathlib.Path] Path to the input data in JSON format.

CHAPTER

SIX

RELEASE NOTES

This document contains the release notes for the project.

6.1 0.2.0 - March 03, 2026

Dependencies

Bump actions/checkout from 6.0.1 to 6.0.2	#51
Bump actions/setup-python from 6.1.0 to 6.2.0	#52
Bump ansys/actions from 10.1.4 to 10.2.5	#81

Documentation

Website Documentation	#82

Fixed

Edited the splash screen	#70
Remove Deletion of Top Level Parent Connection	#76
Remove AEDT Design title in toolkit Help Menu	#78
Deprecated arguments in generate_unique_project_name	#80
Delete Button added to delete row types Layers and Sides	#86

Maintenance

Update CHANGELOG for v0.1.0	#65
Bump version 0.2.0dev and update to year 2026	#66
Add new review changes	#71
Review of the backend code base	#85

Miscellaneous

Assign_matrix	#84
Changes related to UI / frontend logic	#88
Remove AEDT timeout	#93

6.2 0.1.0 - January 30, 2026

Added

JSON Example Files Added	#5
Settings Group Added to LHS UI Column	#186
Add Support for Bespoke and Reference Core Dimensions	#196
update json format	#210
BE tests improvement	#214
UI Save Button	#215
Add Connections to Data Exchange	#231
Add FE Tests	#232

Dependencies

Bump actions/setup-python from 6.0.0 to 6.1.0	#212
Bump softprops/action-gh-release from 2.3.3 to 2.5.0	#217
Bump actions/checkout from 5.0.0 to 6.0.1	#218
Bump actions/download-artifact from 5.0.0 to 7.0.0	#223
Bump actions/upload-artifact from 4.6.2 to 6.0.0	#224

Documentation

Remove Fixed Issues from README.rst	#11
Correct README.rst syntax	#13
Fix Help Menu Title "PyETK Transformer Toolkit"	#18
Improved backend API documentation	#33
Update badge link in README.rst	#44

Fixed

Fixed pyaedt arguments	#22
Backend model json_version typo	#49
Update backend properties, one property at a time	#54
Fixed layer spacing	#60
Rename create_transformer to create_model	#188
Fix load in side 1	#191
Fixes BE issues	#197
Frequency unit conversion	#205
Read from and Write gui_properties	#216
import fe_properties as properties	#229

Maintenance

Update license to Apache-2.0	#2
Clean doc and tests jobs	#8
Refactor into multiple workflors	#21
Update project's root level files	#24
Add missing checkout and wheelhouse	#56
Add missing job dependency in release job	#64
update CHANGELOG for v0.1.0a0	#178
bump 0.2.dev0	#179

Miscellaneous

Update Save As button text	#45
Update main_page.py	#46
PostProcessing and new HTTP request through /create_model	#184
remove/modify PyAEDT from toolkit naming convention	#189
Frontend Data and DataBase Manager	#228
Connections Definitions and Minor FE Fixes	#230

Test

Add additional bobbin unit tests	#6
Add additional data_manager unit tests	#7
Add circuit test	#9
Improved core tests	#14
UI Core with and without Airgap	#15
Add tests to FE data manager	#16

CHAPTER

SEVEN

WHAT IS THIS LIBRARY?

PyETK leverages PyAEDT methods and best practices to develop electronic transformers. It brings numerous advantages like enhancing efficiency, consistency, and collaboration in the creation and development of AEDT magnetic component models.

PYTHON MODULE INDEX

a

ansys.aedt.toolkits.electronic_transformer.backend.api, ??